

Sapienza

Università di Roma

Ingegneria dell'Informazione, Informatica e
Statistica

Laurea Triennale

Dipartimento di Informatica

Algoritmi II

Appunti a cura di
Lorenzo Ferraro

27 settembre 2026

© 2026

Distribuito sotto licenza CC BY-NC-SA 4.0

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.it>

Informazioni e Segnalazioni

Questi appunti raccolgono i contenuti del corso di *Algoritmi II*, tenuto nell'ambito del corso di laurea in Informatica presso "La Sapienza".

Il materiale viene mantenuto e aggiornato periodicamente sul mio repository personale github.com/TonnoArcobaleno/uni-notes, dove è possibile consultare la versione più recente e proporre correzioni o integrazioni.

Questi appunti non sostituiscono in alcun modo i libri di testo consigliati dai docenti o la partecipazione alle lezioni in aula, ma nascono con l'obiettivo di essere un supporto e agevolare lo studio e ripasso.

Segnalazioni

Trattandosi di appunti scritti e rivisti da uno studente, è possibile che contengano imprecisioni, refusi o passaggi poco chiari. Se ne individui uno, la segnalazione è benvenuta e può avvenire in due modi:

- aprendo una **Issue** sul repository indicato sopra, indicando il capitolo o la sezione interessata e descrivendo brevemente il problema riscontrato;
- scrivendo direttamente a **ferraro.notes.issues@gmail.com**, specificando nell'oggetto il documento a cui ci si riferisce e, nel testo, il punto esatto e la natura dell'errore (è il canale preferito, perché più facile che io mi accorga della segnalazione in breve tempo).

Poiché il documento viene revisionato periodicamente, prima di segnalare un problema conviene verificare che non sia già stato corretto nell'ultima versione disponibile sul repository.

Indice

1	Grafi e Algoritmi sui grafi	1
1.1	Introduzione ai grafi	1
1.2	Problema dei ponti di Konisberg	2
1.3	Rappresentare i grafi	3
1.4	Cicli	5
1.4.1	Problema sui cicli	6

Capitolo 1

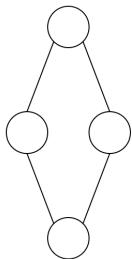
Grafi e Algoritmi sui grafi

1.1 Introduzione ai grafi

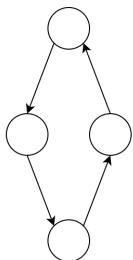
Definizione 1.1 – Grafo

Un **grafo** è una coppia ordinata $G = (V, E)$ dove V è l'insieme dei **nodi** (o vertici), mentre E è l'insieme degli **archi** che collegano due vertici.

In questi appunti guarderemo a grafi senza loop (ossia senza archi che collegano un nodo a sè stesso) e senza archi paralleli (ossia grafi che non hanno due o più archi uguali).



Questo è l'esempio più classico di grafo: i nodi sono i pallini, che sono collegati da delle linee che rappresentano gli archi.



Gli archi possono avere anche un orientamento, e se tutti gli archi lo hanno diciamo che il grafo è **orientato**, come il grafo qui a sinistra. In questo caso è possibile avere due archi differenti tra due nodi x e y , ma devono avere la freccia invertita (ossia uno deve portare da x a y e l'altro da y a x).

Proprietà 1.1 – Archi

Preso un arco f da x a y , diciamo che

- f è **incidente** a x e y
- x è **adiacente** a y ($x \sim y$)

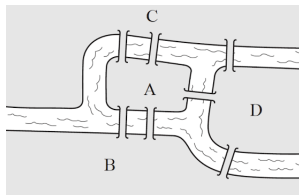
Definizione 1.2 – Grado di un vertice

Il **grado di un vertice** x ($\deg(x)$) è il **numero di archi incidenti** ad esso.

Nei due esempi visti prima, il grado di ogni vertice era 2.

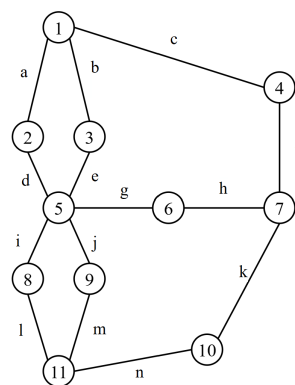
1.2 Problema dei ponti di Konisberg

Il problema dei ponti di Konisberg è uno dei problemi logico-matematici più famosi.



La domanda è semplice: con la seguente disposizione di ponti, è possibile attraversarli tutti una volta sola e tornare al punto di partenza?

Questo classico quesito è risolvibile usando i grafi. Infatti, possiamo schematizzarlo in questo modo:

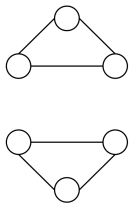
**Definizione 1.3 – Passeggiata**

Una **passeggiata** in un grafo è una **sequenza** del tipo $v_0 e_0 v_1 e_1 \dots v_n e_n$, dove v_i e v_{i+1} indicano due vertici collegati dall'arco e_i .

Una passeggiata è detta **chiusa** se $v_0 = v_n$.

Una passeggiata possibile nel grafo qui sopra ad esempio è "1 a 2 d 5 i 8 l 11 n 10 k 7".

Per risolvere il problema dei ponti di Konisberg bisogna saper rispondere alla domanda "Quand'è che esiste una passeggiata chiusa in un grafo che contiene tutti gli archi presenti una volta sola?". La risposta è che **qualsiasi grafo con almeno un nodo x tale che $\deg(x) \% 2 = 1$** (ossia il cui grado è dispari) **non ammette una passeggiata di questo tipo**. Nel nostro caso, quindi, il problema è irrisolvibile: infatti sia il nodo 1, 5, 7 e 11 hanno un grado dispari. Ma se fossero tutti pari? Non è comunque detto.



In questo caso, infatti, nonostante tutti i nodi siano di grado 2 quindi pari, la risposta al problema è comunque no. È infatti comunque necessario che il grafo sia **connesso**.

Definizione 1.4 – Grafo connesso

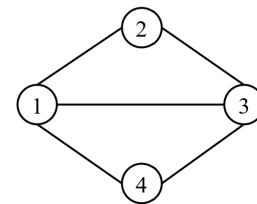
Un grafo si dice **connesso** se per ogni coppia di vertici x e y esiste una passeggiata che porti da x a y .

Proposizione 1.1

Esiste una passeggiata chiusa che contiene tutti gli archi del grafo una sola volta $\iff$ ogni vertice ha grado pari e il grafo è chiuso.

1.3 Rappresentare i grafi

Vedremo ora due modi differenti per rappresentare questo grafo:



Definizione 1.5 – Matrice di adiacenze

Si prende una matrice M di dimensione $n \times n$ (dove $n = \#V$) tale che

$$m_{ij} = \begin{cases} 1 & \text{se } x \sim y \\ 0 & \text{altrimenti} \end{cases}$$

Nel nostro caso quindi abbiamo una matrice del tipo

$$M = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

Questa matrice, dato che non abbiamo loop nei grafi, avrà sempre la diagonale principale composta solo da 0.

Proposizione 1.2

Se il grafo non è diretto, la matrice di adiacenze sarà **speculare**.

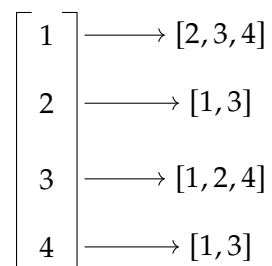
Proprietà 1.2 – Costi computazionali matrice di adiacenze

- **Memorizzazione del grafo:** $\Theta(n^2)$ (spaziale)
- **Visita del grafo:** $\Theta(n^2)$
- **Verificare se $i \sim j$:** $\Theta(1)$
- **Preso un nodo x , calcolare $\text{deg}(x)$:** $\Theta(n)$

Definizione 1.6 – Lista di adiacenza

Si prende un'array di dimensione n , e a ogni posizione i dell'array si punta ad una **lista puntata con tutti i vicini del nodo i -esimo**.

Nel nostro caso abbiamo quindi uno schema del tipo

**Proprietà 1.3 – Costi computazionali lista di adiacenze**

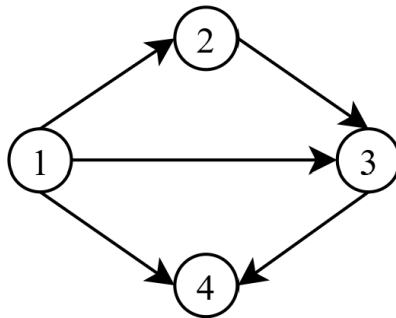
- **Memorizzazione del grafo:** $\Theta(n + m)$ (spaziale), dove $m = \#E$
- **Visita del grafo:** $\Theta(n + m)$
- **Verificare se $i \sim j$:** $O(\min(\text{deg}(i), \text{deg}(j)))$
- **Preso un nodo x , calcolare $\text{deg}(x)$:** $\text{deg}(x)$

In realtà, per la memorizzazione e la visita avremmo un costo pari a $\Theta(n + \sum_{v \in V} \text{deg}(v))$, ma dato che $\Theta(n + \sum_{v \in V} \text{deg}(v)) = \Theta(n + 2m) = \Theta(n + m)$ non abbiamo problemi.

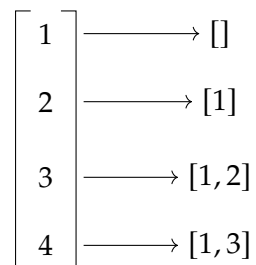
Per i grafi diretti, invece, ci sono 3 possibilità:

- 1) Le liste di adiacenze hanno solo i nodi in entrata
- 2) Le liste di adiacenze hanno solo i nodi in uscita
- 3) Ci sono due liste di adiacenze, una per i nodi in entrata e una per quelli in uscita

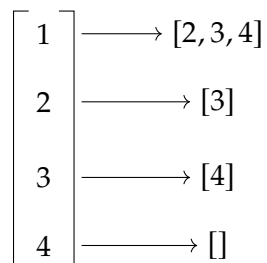
Tutti e 3 i modi sono perfettamente funzionanti. Vediamo con un esempio perché.



Se usiamo le liste di adiacenza con solo i valori in entrata, allora avremo uno schema del tipo



mentre quelle con solo valori in output saranno del tipo



In entrambi i casi, però, non stiamo perdendo informazioni sul grafo: infatti è possibile da queste liste ricreare il grafo da cui vengono senza alcun problema.

1.4 Cicli

Definizione 1.7 – Cammino

Un **cammino** è una passeggiata che **non ripete i vertici**.

Proposizione 1.3

Presi due nodi x e y , esiste un cammino da x a y $\iff$ esiste una passeggiata da x a y .

Dimostrazione

1° Esiste un cammino da x a $y \implies$ esiste una passeggiata da x a y

È ovvio, dato che un cammino è una passeggiata. ■

2° Esiste un cammino da x a $y \iff$ esiste una passeggiata da x a y

Prendiamo la passeggiata di lunghezza minima tra x e y , e ipotizziamo per assurdo che non sia un cammino. Dunque sicuramente esiste un nodo che viene ripetuto, ossia esistono due indici $i < j$ tali che $v_i = v_j$. Possiamo allora tagliare il percorso che porta da v_i di nuovo a se stesso, dato che stiamo inutilmente allungando il giro. Ma così staremmo accorciando la passeggiata, che abbiamo detto essere di lunghezza minima. Abbiamo quindi trovato un assurdo, dunque abbiamo trovato che la passeggiata più corta è un cammino da x a y . ■

Definizione 1.8 – Ciclo

Un **ciclo** è un grafo connesso dove ogni vertice è di grado 2.

1.4.1 Problema sui cicli

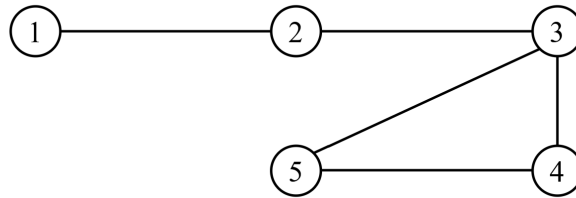
"Dato un grafo dove ogni nodo x ha $\deg(x) \geq 2$, trovare un ciclo". Proviamo a studiare un algoritmo per risolvere questo problema.

Scegliamo un nodo di partenza x . Camminiamo lungo il grafo spondandoci sempre su un vicino diverso dal quale siamo arrivati (il che è possibile dato che $\deg(x) \geq 2$). Prima o poi troveremo un vicino già visitato. In quel momento il nodo su cui ci troveremo avrà due vicini già visitati, il che significa che siamo arrivati ad un ciclo.

```
1 def trovaCiclo(G):
2     L = lista vuota
3     x = nodo qualsiasi con grado 2
4     L.append(x)
5     current = x
6     next = x.neighbors[0]
7     while(next not in L):
8         L.append(next)
9         if next.neighbors[0] == current:
10             u = next.neighbors[1]
11         else:
12             u = next.neighbors[0]
13         current = next
14         next = u
15     while L[0] != next:
16         L.remove(L[0])
17     return L
```

L'ultimo while è fondamentale perché rimuove tutti i nodi che non fanno parte del ciclo.

Vediamo con un esempio grafico perché questo algoritmo funziona, studiando il grafo



Iniziamo con $L = [2]$, $current = 2$ e $next = 3$. Dopo il primo ciclo avremo $L = [2, 3]$, $current = 3$ e $next = 4$. Dopo il secondo ciclo avremo $L = [2, 3, 4]$, $current = 4$ e $next = 5$. Dopo il terzo ciclo avremo $L = [2, 3, 4, 5]$, $current = 5$ e $next = 3$. Dato che ora $next$ è in L , usciamo dal ciclo. Nel secondo while eliminiamo 2, e dunque rimane $L = [3, 4, 5]$, che è proprio il ciclo nel grafo.

Questa versione, però, non è la più efficiente. Infatti, controllare che un elemento non sia in una lista non ordinata ha un costo $O(n)$, dunque, dato che il primo while è eseguito n volte, arriviamo ad un costo in $O(n^2)$. Vediamo un'altra versione più efficiente.

```

1 def trovaCicloEfficiente(G):
2     L = lista vuota
3     Vis = [0] * n
4     x = nodo qualsiasi con grado 2
5     L.append(x)
6     Vis[x] = 1
7     current = x
8     next = x.neighbors[0]
9     while Vis[next] == 0:
10         L.append(next)
11         Vis[next] = 1
12         if next.neighbors[0] == current:
13             u = next.neighbors[1]
14         else:
15             u = next.neighbors[0]
16         current = next
17         next = u
18     while L[0] != next:
19         L.remove(L[0])
20     return L
  
```

In questo caso usiamo quindi un array di supporto dove controlliamo se un nodo è già stato visitato o meno, riducendo il controllo della condizione del while da $O(n)$ a $O(1)$, abbassando

quindi il costo temporale a $O(n)$. Il costo spaziale invece rimane identico in entrambi i casi: $O(n)$.